- React is a library, it is a single page application
- Library – collection of modules
- Modules are predefined codes which contains class and function
- All modules are stored in node

React Installation

1. npm create vite@latest
2. project name
3. package name(optional)
4. framework – React
5. variant – javascript
6. npm install ---yes
7. Kill the server by ----q+enter
8. Change the directory ---cd project name
9. Open vs code--- code .
10. Start the server --- open terminal --- npm run dev
11. To open in browser --- o+enter

React Folder Structure

- **Node module** All files/modules necessary for react will be present inside node module
- **Public** is a place to store static files like images, video, audio etc
- **Src** – is a place where we write the actual code/ react logics
- **Package.json** - is a configuration file of the project, where we get all information about the project like project name, package name, version, what are the script files, what are the production dependencies, dev dependencies etc used in the project.
- **Index.html** - In react only one html file is there that is index.html, here only all the logics will be appended
- Dependencies are external sources, DevDependencies are use for development purpose

ELEMENT

It is a fundamental building block which is used to provide content and structure to react, In react elements are considered as object

We can create element in 2 ways

1.React.createElement()

This will accept 3 arguments, element name, attribute object, and displaying content for paired element

For unpaired element it will take 2 arguments element name and attribute object

EX: MAIN.JSX

```
import React from "react";
import { createRoot } from "react-dom/client";
const element = React.createElement(
 "h1",
 { id: "title" },
 "Welcome"
);
console.log(element);
createRoot(document.getElementById("root")).render(element);
```

Code	Responsibility
`<div id="root"></div>`	Creates the empty container (in `index.html`)
`document.getElementById("root")`	Finds that container
`createRoot(...)`	Makes React manage that container
`.render(element)`	Displays the React element inside the container

Interview Tip ⋆

If an interviewer asks:

What does `createRoot()` do?

A good answer is:

"`createRoot()` creates a React root for an existing DOM element. It tells React to manage and render the application inside that DOM element."

2.JSX

It stands for java script and xml, It will allow to write html syntax inside java script

RULES

1. Always jsx should return only one parent element, inside parent element we can create n number of child element(If you want to create multiple elements we can achieve using 2 ways **1.<></>-Empty tag, 2.<React.Fragment>-Fragment tag**) [because React expects a single root element to represent the entire UI structure. If multiple sibling elements are written without a parent, React cannot determine a single root and throws an error.]
2. All the element name should be written in lower case
3. Attribute name should be camel case
4. Some attribute name has been changed **for-htmlFor, class-className**
5. All the tags must be closed either by self closing tag or by separate closing tag

FRAGMENT

It is a special component in react which helps to wrap multiple elements without adding any extra node in actual DOM

2 ways we can create

1. <React.Fragment></React.Fragment> - here we can provide key attribute

2. <></> - we can't provide any attribute

⋆Very Important Interview Question

Interviewer: What is the difference between `<div>` and `Fragment`?

** **	**Fragment**
Creates an actual HTML element.	Does not create any HTML element.
Appears in the DOM.	Does not appear in the DOM.
Can affect CSS/layout.	Keeps the DOM clean.

EXPRESSION

It is used to access variable and perform operation along with JSX

Rules of Expression

1. We can't declare variables inside expression but we can access variable
2. We can't use if, else, switch case inside expression but we can use ternary operator and (&)operator
3. we can't use for, while loop for iteration, we need to use array higher order/iteration methods like map or filter
4. We can declare and call only IIFE function, other function we can only declare or call

COMPONENT

- It is self contain piece of reusable code, which helps to create specific part of the user interface
- Self contain means it will have own logic and own css
- In react we can create component in 2 ways
    1. Function based component
    2. Class based component

FUNCTION BASED	**CLASS BASED**
Simple syntax and structure	Complex syntax and structure
We don't need component class to create component	We need component class to create component
Stateless component means, it will not have own state, to get the state we need to use state hook	State full component. It has own state
There is no inbuild life cycle methods, we can get life cycle methods by using use effect hook	Inbuild life cycle methods are available

RULES OF COMPONENT

1. Component name should be title case
2. Return type should be jsx
3. We need to call component by using tags

KEYS

It is a special prop in react to uniquely identify the elements. It helps react to efficiently update, add or remove the elements

RULES

1. It should be unique, no two elements should have same key

PROPS

- Props stands for property
- It is unidirectional used to transfer the data from parent component to child component not child to parent
- While sharing the data we have to share like jsx attributes
- While receiving the data
  1. In function based we need to receive with a parameter that parameter is props object
  2. In class based we need to use this.props to get the data. And this.props is also object
- if we share elements from parent component, we have to use children property to get the elements

  EX: for variable and data
  <parent name="messi" age="39"></parent>
  For elements:- this h1 is considered as children
  <parent><h1></h1></parent>

EVENTS

- Events are action or interaction happen between user and web page
- In react all the events are treated as synthetic events, that means in all browser all the events will behave in same manner

HOW REACT DIFFERENT FROM JS EVENT

- Event name should be camel case
  Eg: onclick is wrong | onClick is correct
- Event handler should be passed not called
  Eg: onclick=" " is wrong | onClick={} is correct
- Event handler should be passed inside expression not in string
  Eg: onclick="abc()" is wrong | onClick={abc} is correct
- If we want to execute in capturing phase we can provide as jsx attribute with the suffix of capture

Eg: onClickCapture | onMouseCapture

CONDITIONAL RENDERING

It is the process of displaying different jsx element or component based on condition

STATE (Important)

- It is a dynamic data. It means whenever it changes it will re-render the component.
- Re-render means re call component with update state value
- We can create state in 2 ways

FUNCTION BASED COMPONENT BY USING USESTATE HOOK

- useState is a hook(function), it will accept one argument that is current state value. And it return one array and that array has 2 elements
- First element is current state and second element is set state function
- If we want to update the state we need to update by using setState function only
- Here state can be any data type

CLASS BASED COMPONENT BY USING STATE OBJECT

- We can declare state by using state object and name must be state only and state data-type should be object.
- We can update the state by using **this.setState()** function

```
EX:
state={count:0}
abc=()=>{
this.setstate({count.this.state.count+1})
}
Render(){
Return <div>
<h1>{this.state.count}</h1>
<button onClick = {abc}> increase</button>
</div>
}
```

VIRTUAL DOM/ RECONCILATION/ DIFFING/ DIFFING ALGORITHM (Important)

- It is an object

- It is a light weight clone of real DOM

WORKING PROCESS

- In the initial render our jsx will be converted to js by using **babel engine**
- Based on js one new virtual DOM will be created, and based on virtual DOM one **actual DOM will be created**
- Based on actual DOM UI will be created/painted
- Whenever state or props changes it will re-render(load) the component
- Again our jsx will be converted to js by using babel engine
- Based on js one new virtual DOM will be created, now our old virtual DOM will be compared with new virtual DOM, this comparing process is called as **diffing**
- Based on the comparison **actual DOM will be updated,** this updating process is called as **reconciliation**
- Based on the actual DOM UI will be created/painted
- Previously the reconciliation process happened in synchronized(block next execution) way
- After react 16(version) they introduced react fibre, it is the reconciliation engine and it will do the task in asynchronous(doesn't block next execution) way.

DIFFERENCE BETWEEN STATE AND PROPS (Important)

STATE	PROPS
In class based component it is an object, in function based component it can be any data	it stands for properties, it is an object
To create dynamic UI	It is used to transfer the data
It is mutable	It is immutable
Data belongs to own component	Data belongs to parent component
it will also re-render the component	It will re-render the component

HOOKS

- It is a function and it will add extra features to function based component
- Common hooks are useState, useRef,useContext,useReduce,useEffect,useMemo, useCallback, useLayoutEffect, useInsertionEffect

RULES OF HOOK

- All the hooks should start with the name use
- All the hooks should be called inside function based component
- All the hooks should be called at top level of the component. It should not be under any conditional statement, or loops or callback function.

REF

- It stand for reference, It is used to target dom elements directly
- It is mutable data
- It will store the data across all renders without causing any re-renders.
- It is also used to create uncontrolled forms

CONTROLLED FORMS

In controlled forms, form data will be handled by state

UNCONTROLLED FORMS

Here form data will be handles by ref, it will directly access the actual dom

CONTEXT API

It is used to transfer the data between the components, it is alternative props. It will avoid probs drilling

PROBS DRILLING

- Sharing data in nested components is call as probs drilling
- Like app to parent, parent to child, child to grandchild with the help of props

THERE ARE THREE STEPS

1. CREATE CONTECT

Create context is a function, when we call this function it will return one object. That object should be exported

EX: export let myContext= createContext()

2. PROVIDE CONTEXT

By using the object which is returned by createContext we need to call provide component. Which data we need to pass to another component must be shared as a value attribute to provider component

EX: return <myContext.provider value=" "></myContext.provider>

3. CONSUME CONTEXT

The exported object needs to be imported here, and the object needs to be passed as an argument to useContext hook. This useContext hook will return the value of provide component.

EX: import {myContext} from 'filename'
let data = useContext(myContext)

LIFE CYCLE METHODS

- These are the special methods which will be executed in different stages of components
- There are three phases
  1.Mounting phase – component added to dom tree
  2.Updating phase – re-rendering the component
  3.Unmounting phase – component removed from dom tree

LIFE CYCLE METHODS SYNTAX

1.Mounting- component is added to dom tree

Syntax:

```
useEffect(()=>{},[])
```

2.updating-whenever state updated it will be executed

Syntax:

```
useEffect(()=>{},[state1,state2])
```

3.unmounting – when component removed from dom tree

Syntax:

```
useEffect(()=>{
return ()=>{}
},[])
```

MOUNTING PHASE

1. constructor – this is the best place for declaring the state

2. static getDerivedStateFromProps() – it will run in all re-rendors, state will be updated based on props
3. render() – execute in all renders, returns jsx
4.ComponentDidMount()- used for handling API request

UPDATING PHASE

1.static getDerivedStateFromProps() – it will run in all re-renders, state will be updated based on props
2.shouldComponentUpdate() – it will determine whether component should render or not
3.render()-execute all the renders, return jsx
4.getSnapShotBeforeUpdate() – used for scroll position
5.ComponentDidUpdate() – used for handling api request

UNMOUNTING PHASE

1.ComponentwillUnmount() – used for cleanup function such as class interval,ClassTimeOut()

USEEFFECT

- It is used to handle side effects of react and also to achieve life cycle methods in function based component
- Side effects means dom events, timer functions, api request(all the asynchronous operations are considered as side effects)

useEffect will accept 2 arguments , first one is call back function, second one is dependency array. If dependency array empty it will be executed only once when it is added to dom tree. If dependency array having state it will be executed whenever state updated. For unmounting we need to return one function that function will be executed when component is removed.

USE LAYOUT EFFECT:

It is for synchronous code just before ui is painted on the screen. Mostly used for scroll position and animations.

USE INSERTION EFFECT:

It is also synchronous code . It will be executed just before actual dom is updated .

Memorization:

It is an optimization technique Which we avoid unnecessary re – rendering or child component, and functions decleration and heavy mathematical calculation we can achieve it in three ways .

MEMO:

It is higher order component . It will accept another component as an argument. It will avoid unnecessary re-rendering of child component. It will re-render only when props is changed .

USEMEMO:

It is which will capture value of two function call and it avoids unnecessary repeated call of function in re-render. It will avoid recalculation of heavy mathematical calculation. It will calculate again only when dependency array state changes.

SYNTAX:

```
function sum(a,b){}

Let sumValue=useMemo(()=>{

Return sum(10,5)

},[])
```

USECALLBACK:

It is a hook which will capture the function declaration and avoid unnecessary repeated declaration of function while re rendering

USE REDUCER

- It is an alternative to state, whenever we need complex state with multiple action, in those scenarios we need to use useReducer hook
- It accepts 2 arguments first one is reducer function and second one is current state value.
- It returns one array and that array has 2 elements, current state and dispatch function
- This dispatch function accepts one argument that is action
- Whenever we call the dispatch function, it will call the reducer function with action what we provide
- Reducer function have two parameter current state and action

- In reducer function only we need to write logic for state in different action
- Syntax
  Outside component:
  Function reducer(state,action){}
  Inside component
  let [state,dispatch]=useReducer(reducer,state)
  dispatch(action)

ROUTER

- It is used to navigate different pages or component without re-loading the browser
- In react to get router we need to install react-router-dom(npm install react-router-dom
- To set up router we need to import create browser router function and routerprovider component from react-router-dom
- **Create browser router-** it is used for configuring router
- **RouterProvider –** to make router accessible for all the component. Or to make router available for entire app.
- Create browser user accept one argument that is array. That array is considered as routers
- Inside array we can pass n number of objects, those objects are considered as route
- in object important properties are
  1.Path : to provide url end point
  2.element: to display the element or component

ERROR ELEMENT

It is used to display whenever any error appears with respect to routes, It will render only whenever any error appears

CHILDREN

It is an array of object, it will contain child routes which will render along with parent route

FOR NAVIGATION

We can use Link component or useNavigate() hook

For Link we need to provide path into attribute

<Link to=”path”></Link>

UseNavigate return one function in that function we need to pass path

Let navigate=useNavigate()

Navigate(path)

FOR DYNAMIC ROUTING

The path should start by : and variable name, to access path we need to use useParams hook, this hook will return one object and that variable name will be given as property here.

Path:”id”

Let {id}=useParams()

CODE SPLITTING AND LAZY LOADING

- Code splitting is a optimization technique which will divide larger file(bundles) code to multiple small file(bundles).
- Those smaller files will be downloaded whenever it needed by using lazy function.
- Without lazy all files will be downloaded at once whether it is needed or not
- But with lazy function it will download only when it is required
- Suppose if it is takes more time to download during that time to display some message we need suspense component.
- In the suspense component we need to pass callback attribute to display message
- **SYNTAX:**
  let Home = lazy(()=>{
  return import(“./Home”)
  })
  <Suspense fallback=”…loading”><Home></Home></Suspense>

WHAT IS REACT

It is a javascript library which is used to create single page application

FEATURES OF REACT

- It supports virtual DOM for faster re-renders
- It is a component based architecture
- It supports jsx to display html syntax
- It is uni-directional, it means it will transfer the data from parent component to child.
- It is declarative - we don't have to define for most of the feature, we will use the inbuild feature(logic)
- It supports hooks
- It supports routing by using react-router-dom.

React Tools: (MENTION IN RESUME)

@tanstack/react query – for server side state management

React-hot-toaster - for notification

React-router-dom – for routing

Axios – for http request

STATE MANAGEMENT

UseReducer, useState, useContext- for client side state management

@tanstack/react query – for server side state management